

Cambios realizados en feature/frontend-infraestructura

Contraste técnico con el documento “Arquitectura General y Stack Tecnológico”

Proyecto: Sistema de Legajos — Bomberos Voluntarios

Rama examinada: feature/frontend-infraestructura

Base de comparación: 98dad46 — feature/frontend-planes-navegacion

Último commit: 58dcc3e

Fecha de verificación: 22 de julio de 2026

Alcance: frontend React; revisión de Git, código y build de producción.

Resumen ejecutivo

La rama incorpora cuatro bloques principales que coinciden con el documento entregado:

- 1. RBAC visual, sesión en `sessionStorage` e integridad referencial preventiva.
- 2. Refactor de paneles mediante `TablaPrincipal`, componentes y hooks reutilizables.
- 3. Unificación de formularios y flujos guiados a pantalla completa.
- 4. Code-splitting por rutas y separación manual de dependencias externas.

Indicador	Resultado comprobado
Commits propios de la rama	4
Cambio total frente a la base	176 archivos; 20.687 inserciones y 15.860 eliminaciones
Build actual	Correcto; 233 módulos; 1,96 segundos
Bundle de entrada	14,39 kB; 4,49 kB comprimido
Vendor React	251,90 kB; 80,97 kB comprimido
Chunk mayor de negocio	ConfigDocumentos: 75,16 kB
Advertencias de tamaño	Ninguna en la verificación actual

Conclusión importante: el RBAC de esta rama es una demostración de interfaz. El login genera un token simulado y el backend no valida esos permisos. Por eso no debe presentarse como seguridad completa de producción.

1. Historial de cambios de la rama

Commit	Propósito	Magnitud
6275b4f	RBAC con sessionStorage, validaciones de integridad y separación masiva en componentes/hooks.	154 archivos; +19.962 / -14.856
2158a75	Migración de paneles a TablaPrincipal y unificación adicional de altas guiadas.	95 archivos; +4.785 / -5.380
bd68a0c	Unificación visual de formularios guiados a pantalla completa.	11 archivos; +1.116 / -828
58dcc3e	Code-splitting con React.lazy, Suspense y manualChunks.	2 archivos; +97 / -69

La suma por commit puede contener líneas modificadas varias veces. La diferencia final contra la base es la cifra consolidada de 176 archivos.

2. Arquitectura y stack tecnológico

COMPROBADO

El package.json confirma el stack descrito:

Tecnología	Versión declarada	Uso
React	^19.2.7	Componentes y estado de interfaz.
React DOM	^19.2.7	Renderizado en navegador.
React Router	^8.0.1	SPA y navegación.
Vite	^8.1.0	Servidor de desarrollo y build.
Tailwind CSS	^4.3.1	Estilos utilitarios.
@tailwindcss/vite	^4.3.1	Integración nativa con Vite.
Oxlint	^1.69.0	Lint mediante npm run lint.
Lucide React	^1.21.0	Iconografía.
React Hot Toast	^2.6.0	Notificaciones.

Identidad visual

Los componentes utilizan rojo institucional, fondos slate, tarjetas blancas, bordes suaves y estados hover. Se observa de forma consistente el uso de clases como `bg-red-700`, `hover:bg-red-800`, `bg-slate-50` y `bg-slate-100`.

Archivos de referencia

- `frontend/package.json`
- `frontend/vite.config.js`
- `frontend/src/index.css`
- Componentes y páginas bajo `frontend/src`

La elección de React frente a Next.js es una justificación arquitectónica, no un cambio específico de estos cuatro commits. El repositorio ya era una SPA antes de esta rama.

3. Patrón TablaPrincipal

IMPLEMENTADO

Se creó frontend/src/components/TablaPrincipal/TablaPrincipal.jsx para centralizar búsqueda, ordenamiento, paginación y renderizado de filas.

Comportamientos comprobados

- **Búsqueda global:** filtra sobre propiedadesFiltro o sobre las columnas configuradas.
- **Ordenamiento:** usa useMemo y localeCompare(..., "es").
- **Valores anidados:** admite valorOrden(item) por columna.
- **Paginación:** usa slice y permite 10, 25 o 50 filas.
- **Reinicio de página:** vuelve a página 1 cuando cambia la búsqueda o el tamaño.
- **Acciones:** acepta una función accionesPorFila.

```
useEffect(() => {
  setPaginaActual(1);
}, [busqueda, filasPorPagina]);

const comparacion = valorA.localeCompare(valorB, "es", {
  sensitivity: "base",
  numeric: true,
});

const inicio = (paginaActual - 1) * filasPorPagina;
return datosProcesados.slice(inicio, inicio + filasPorPagina);
```

Paneles que consumen TablaPrincipal

- AulasPanel
- ComisionesPanel
- PersonasPanel
- RangoPanel
- SedesPanel
- TipoDocumentoPanel
- TiposSedesPanel

Componentes auxiliares

- components/filtros/filtroGlobal.jsx
- components/filtros/filtroTablas.jsx

4. Altas guiadas y wizards

IMPLEMENTADO

Los flujos complejos se dividieron en pasos independientes, componentes visuales y hooks de lógica. Los tres wizards principales son:

Flujo	Página principal	Hook/componentes relevantes
Alta de persona	AltaPersonaWizard.jsx	useAltaPersonaWizard.js y componentes StepPersona, StepLegajo, StepDatosLegajo, StepUsuario y StepResumen.
Alta de plan	AltaPlanWizard.jsx	useAltaPlanWizard.js y pasos Plan, Asignatura, Condiciones, Correlativas y Resumen.
Alta de comisión	AltaComisionWizard.jsx	useAltaComisionWizard.js y pasos Comisión, Asignaturas, Autoridades y Resumen.

Persistencia escalonada

El alta de Plan guarda primero la entidad base. Con el identificador resultante habilita la carga de asignaturas, condiciones y correlativas. El flujo mantiene un estado de paso y culmina en un resumen.

```
const [pasoActual, setPasoActual] = useState(1);

async function guardarPlan(event) {
  // valida y crea el plan base
  // conserva el id generado
  // avanza al siguiente paso
}
```

Mejoras visuales

El commit bd68a0c ajustó once archivos de formularios y paneles para unificar la experiencia a pantalla completa, especialmente en infraestructura, personas y configuraciones.

En el árbol actual coexisten algunas familias de componentes con nombres Paso... y Step.... Esto refleja etapas sucesivas del refactor y conviene tenerlo presente para evitar duplicación futura.

5. Integridad referencial preventiva

IMPLEMENTADO EN UI

Los paneles consultan relaciones existentes antes de habilitar acciones destructivas. El ejemplo del documento está presente en `TipoDocumentoPanel.jsx`:

```
const estaAsociado = (id) =>
  personas.some(
    (persona) => persona.td_id === id || persona.tipoDocumentoId === id
  );

<button
  disabled={
    !hasPermission(currentUserRole, "eliminar") || estaAsociado(td.id)
  }
  title={
    estaAsociado(td.id)
      ? "No se puede eliminar: tiene personas asociadas"
      : "Eliminar Tipo de Doc."
    }
>
  Eliminar
</button>
```

También se observan controles equivalentes en sedes, aulas, comisiones, asignaturas y otros paneles.

La prevención visual evita peticiones innecesarias, pero la base de datos y el backend deben conservar sus restricciones. El frontend nunca debe ser la única defensa de integridad.

6. RBAC y sesión en frontend

PARCIAL / DEMOSTRATIVO

Matriz centralizada

`authHelper.js` define:

```
ROLE_ADMIN:    ["crear", "editar", "eliminar", "leer"]
ROLE_INSTRUCTOR: ["editar_notas", "leer"]
ROLE_USER:     ["leer"]
```

Distribución del rol

- `AuthContext.jsx` guarda el token en `sessionStorage`.
- `auth.js` decodifica el payload y normaliza distintos nombres de claim.
- `Navbar.jsx` entrega `currentUserRole` a `DesktopNav` y `MobileDrawer`.
- Los paneles usan `hasPermission()` para ocultar altas o deshabilitar edición/eliminación.

Menús

El escritorio y el drawer móvil aplican el mismo criterio. Las opciones de lectura se muestran a los tres roles; las opciones de creación se muestran al administrador.

Limitaciones detectadas

- El login asigna roles según si el texto del usuario contiene “admin” o “instructor”.
- El token utiliza una firma simulada: `mock_signature_key`.
- Los permisos son genéricos y no distinguen recursos.
- Las rutas declaradas en `App.jsx` no están envueltas en una guarda central.
- El backend Flask no valida este token ni estos permisos.
- `sessionStorage` puede manipularse desde el navegador.

La frase “asegura que un usuario no pueda realizar operaciones de escritura” debe matizarse: lo evita desde los controles visibles del frontend, pero no impide una llamada directa a la API.

7. Code-splitting y optimización de build

IMPLEMENTADO Y VERIFICADO

El commit 58dcc3e modifica únicamente App.jsx y vite.config.js.

App.jsx

- Mantiene el Login como importación estática.
- Convierte las páginas administrativas en importaciones dinámicas con lazy().
- Envuelve las rutas con Suspense.
- Muestra “Cargando sección...” mientras llega el chunk.

```
import { lazy, Suspense } from "react";

const Legajos = lazy(() => import("./pages/Legajos"));
const Personas = lazy(() => import("./pages/Personas"));
const Planes = lazy(() => import("./pages/Planes"));

<Suspense fallback=<div>Cargando sección...</div>>
  <Routes>...</Routes>
</Suspense>
```

vite.config.js

- Configura codeSplitting: true.
- Agrupa dependencias React en vendor-react.
- Agrupa las demás dependencias en vendor-helpers.
- Configura el umbral de advertencia en 600 kB.

```
manualChunks(id) {
  if (id.includes("node_modules")) {
    if (id.includes("react")) return "vendor-react";
    return "vendor-helpers";
  }
}
```

8. Resultado actual de producción

Se ejecutó directamente Vite 8.1.0 sobre el estado actual de la rama. La compilación finalizó correctamente:

```
vite v8.1.0 building client environment for production...
233 modules transformed.
✓ built in 1.96s
```

Recurso	Tamaño	Gzip
index JS	14,39 kB	4,49 kB
vendor-react	251,90 kB	80,97 kB
vendor-helpers	5,80 kB	2,73 kB
CSS principal	38,28 kB	7,36 kB
ConfigDocumentos	75,16 kB	10,80 kB
AltaPlanWizard	15,50 kB	4,20 kB
AltaPersonaWizard	12,75 kB	3,86 kB
Personas	8,11 kB	2,63 kB
Comisiones	7,35 kB	2,36 kB
Aulas	9,13 kB	2,87 kB

Comparación con el texto entregado

Afirmación	Estado	Observación
Index reducido a 14,39 kB	Confirmada	Coincide con el build actual.
vendor-react de 251,90 kB	Confirmada	Coincide con el build actual.
Chunks de pantallas independientes	Confirmada	Se generan numerosos archivos bajo demanda.
Sin advertencias de tamaño	Confirmada	No aparecieron advertencias.
Build de 1,34 s	Histórica	La medición actual fue 1,96 s; varía según equipo y caché.
Reducción del 97,6%	Consistente	Comparar 622,20 kB con 14,39 kB da aproximadamente 97,7% para el archivo de entrada; no equivale a reducir en igual proporción todos los bytes iniciales porque también se carga vendor-react.

9. Organización y mantenibilidad adicional

Además de los puntos explícitos del documento, la rama realizó un refactor estructural amplio:

- Separó páginas extensas en componentes pequeños por dominio.
- Extrajo lógica compleja a hooks especializados.
- Creó componentes de formularios, tablas, modales y secciones de edición.
- Separó la navegación de escritorio y móvil.
- Centralizó helpers de formularios y wizards.

Hooks incorporados

- | | |
|-------------------------------|------------------------|
| • useAltaComisionWizard.js | • useContactos.js |
| • useAltaPersonaWizard.js | • useDatosMedicos.js |
| • useAltaPlanWizard.js | • useEditarComision.js |
| • useAutoridadesComision.js | • useEditarPersona.js |
| • useComisionesAsignaturas.js | • useEditarPlan.js |
| • useConfigDocumentos.js | • useNuevoLegajo.js |

Áreas componentizadas

- | | |
|-------------------------|---------------------|
| • AltaComision | • EditarPlan |
| • AltaPersona | • Legajos |
| • AltaPlan | • LegajoRangos |
| • Asignaturas | • LegajoSedes |
| • Aulas | • Personas |
| • AutoridadesComision | • Planes |
| • ComisionesAsignaturas | • PlanesAsignaturas |
| • ConfigDocumentos | • Sedes |
| • Contactos | • TiposDocumentos |
| • DatosMedicos | • TiposSedes |
| • EditarComision | • VerComision |
| • EditarPersona | • VerPlan |

10. Qué está pendiente

SIGUIENTE ETAPA

- Reemplazar el JWT simulado por un token emitido por el servicio de autenticación.
- Definir permisos por recurso: `personas.leer`, `planes.crear`, etc.
- Agregar guardas centralizadas a las rutas React.
- Validar token y permisos en cada endpoint Flask.
- Agregar pruebas automatizadas para la matriz de permisos.
- Revisar componentes duplicados de etapas sucesivas del refactor.
- Medir rendimiento en navegador, no solamente tamaño de archivos.

11. Archivos clave para presentar o reproducir

Tema	Archivos principales
Stack	package.json, vite.config.js, index.css
Tabla genérica	TablaPrincipal.jsx, filtroGlobal.jsx, filtroTablas.jsx
Wizards	AltaPersonaWizard.jsx, AltaPlanWizard.jsx, AltaComisionWizard.jsx, sus hooks y componentes Step/Paso
Integridad	TipoDocumentoPanel.jsx y demás paneles de configuración
RBAC	authHelper.js, auth.js, AuthContext.jsx, Login.jsx
Menús	Navbar.jsx, DesktopNav.jsx, MobileDrawer.jsx
Code-splitting	App.jsx y vite.config.js

12. Conclusión

La rama implementa materialmente los ejes técnicos descritos: componentización, tabla reutilizable, wizards progresivos, controles preventivos, RBAC visual y code-splitting. El resultado de build es reproducible y mantiene el bundle de entrada en 14,39 kB.

La principal precisión necesaria para una presentación es separar **control visual de permisos** de **seguridad real**. La rama evita acciones desde la interfaz, pero aún requiere autenticación y autorización verificadas por el backend.

Resultado general: el documento entregado representa correctamente la dirección del trabajo. Este informe ajusta las métricas al build actual y documenta las limitaciones que deben declararse con claridad.

Informe generado mediante inspección de Git, lectura del código fuente y compilación local. No se modificaron archivos funcionales del frontend.